

SINGLY LINK LIST

```
#include <iostream>
```

```
class Node {
```

```
public:
```

```
    int data;
```

```
    Node* next;
```

```
    Node(int data) {
```

```
        this->data = data;
```

```
        next = NULL;
```

```
    }
```

```
};
```

```
class SinglyLinkedList {
```

```
private:
```

```
    Node* head;
```

```
public:
```

```
    SinglyLinkedList() {
```

```
        head = NULL;
```

```
    }
```

```
    void insertAtBeginning(int data) {
```

```
        Node* newNode = new Node(data);
```

```
        newNode->next = head;
```

```
    head = newNode;  
}
```

```
void insertAtEnd(int data) {  
    Node* newNode = new Node(data);  
    if (head == NULL) {  
        head = newNode;  
    } else {  
        Node* temp = head;  
        while (temp->next != NULL) {  
            temp = temp->next;  
        }  
        temp->next = newNode;  
    }  
}
```

```
void insertAtIndex(int data, int index) {  
    if (index <= 0) {  
        std::cout << "Invalid index" << std::endl;  
        return;  
    }  
    if (index == 1) {  
        insertAtBeginning(data);  
    } else {  
        Node* newNode = new Node(data);  
        Node* temp = head;
```

```

for (int i = 1; i < index - 1; i++) {
    if (temp == NULL) {
        std::cout << "Index out of range" << std::endl;
        return;
    }
    temp = temp->next;
}
if (temp == NULL) {
    std::cout << "Index out of range" << std::endl;
    return;
}
newNode->next = temp->next;
temp->next = newNode;
}
}

```

```

void deleteAtBeginning() {
    if (head == NULL) {
        std::cout << "List is empty, nothing to delete" << std::endl;
        return;
    }
    Node* temp = head;
    head = head->next;
    delete temp;
}

```

```

void deleteAtEnd() {
    if (head == NULL) {
        std::cout << "List is empty, nothing to delete" << std::endl;
        return;
    }
    if (head->next == NULL) {
        delete head;
        head = NULL;
    } else {
        Node* temp = head;
        while (temp->next->next != NULL) {
            temp = temp->next;
        }
        delete temp->next;
        temp->next = NULL;
    }
}

```

```

void deleteAtIndex(int index) {
    if (index <= 0 || head == NULL) {
        std::cout << "Invalid index or list is empty" << std::endl;
        return;
    }
    if (index == 1) {
        deleteAtBeginning();
    } else {

```

```

Node* temp = head;

Node* prev = NULL;

for (int i = 1; i < index; i++) {

    if (temp == NULL) {

        std::cout << "Index out of range" << std::endl;

        return;

    }

    prev = temp;

    temp = temp->next;

}

if (temp == NULL) {

    std::cout << "Index out of range" << std::endl;

    return;

}

prev->next = temp->next;

delete temp;

}

}

```

```

void search(int key) {

    Node* temp = head;

    int position = 1;

    bool found = false;

    while (temp != NULL) {

        if (temp->data == key) {

            std::cout << "Element found at position " << position << std::endl;


```

```

        found = true;
        break;
    }
    temp = temp->next;
    position++;
}
if (!found) {
    std::cout << "Element not found in the list" << std::endl;
}
}

```

```

void display() {
    if (head == NULL) {
        std::cout << "List is empty" << std::endl;
        return;
    }
    Node* temp = head;
    while (temp != NULL) {
        std::cout << temp->data << " ";
        temp = temp->next;
    }
    std::cout << std::endl;
}
};

```

```
int main() {  
    SinglyLinkedList l1;  
    l1.insertAtBeginning(3);  
    l1.insertAtBeginning(6);  
    l1.insertAtBeginning(9);  
    l1.display();  
    l1.deleteAtBeginning();  
    l1.display();  
  
    return 0;  
}
```

DOUBLY LINK LIST

```
#include <iostream>
```

```
class Node {  
public:  
    int data;  
    Node* prev;  
    Node* next;  
  
    Node(int data) {  
        this->data = data;  
        prev = NULL;  
        next = NULL;  
    }  
};
```

```
    }  
};
```

```
class DoublyLinkedList {
```

```
private:
```

```
    Node* head;
```

```
public:
```

```
    DoublyLinkedList() {
```

```
        head = NULL;
```

```
    }
```

```
void insertAtBeginning(int data) {
```

```
    Node* newNode = new Node(data);
```

```
    if (head == NULL) {
```

```
        head = newNode;
```

```
    } else {
```

```
        newNode->next = head;
```

```
        head->prev = newNode;
```

```
        head = newNode;
```

```
    }
```

```
}
```

```
void insertAtEnd(int data) {
```

```
    Node* newNode = new Node(data);
```

```
    if (head == NULL) {
```



```

        head = newNode;
    } else {
        Node* temp = head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
        newNode->prev = temp;
    }
}

```

```

void insertAtIndex(int data, int index) {
    if (index <= 0) {
        std::cout << "Invalid index" << std::endl;
        return;
    }
    if (index == 1) {
        insertAtBeginning(data);
    } else {
        Node* newNode = new Node(data);
        Node* temp = head;
        for (int i = 1; i < index - 1; i++) {
            if (temp == NULL) {
                std::cout << "Index out of range" << std::endl;
                return;
            }

```

```

        temp = temp->next;
    }
    if (temp == NULL) {
        std::cout << "Index out of range" << std::endl;
        return;
    }
    newNode->prev = temp;
    newNode->next = temp->next;
    if (temp->next != NULL) {
        temp->next->prev = newNode;
    }
    temp->next = newNode;
}
}

```

```

void deleteAtBeginning() {
    if (head == NULL) {
        std::cout << "List is empty, nothing to delete" << std::endl;
        return;
    }
    Node* temp = head;
    head = head->next;
    if (head != NULL) {
        head->prev = NULL;
    }
    delete temp;
}

```

```
}
```

```
void deleteAtEnd() {
```

```
    if (head == NULL) {
```

```
        std::cout << "List is empty, nothing to delete" << std::endl;
```

```
        return;
```

```
    }
```

```
    if (head->next == NULL) {
```

```
        delete head;
```

```
        head = NULL;
```

```
    } else {
```

```
        Node* temp = head;
```

```
        while (temp->next != NULL) {
```

```
            temp = temp->next;
```

```
        }
```

```
        temp->prev->next = NULL;
```

```
        delete temp;
```

```
    }
```

```
}
```

```
void deleteAtIndex(int index) {
```

```
    if (index <= 0 || head == NULL) {
```

```
        std::cout << "Invalid index or list is empty" << std::endl;
```

```
        return;
```

```
    }
```

```
    if (index == 1) {
```

```

        deleteAtBeginning();
    } else {
        Node* temp = head;
        for (int i = 1; i < index; i++) {
            if (temp == NULL) {
                std::cout << "Index out of range" << std::endl;
                return;
            }
            temp = temp->next;
        }
        if (temp == NULL) {
            std::cout << "Index out of range" << std::endl;
            return;
        }
        temp->prev->next = temp->next;
        if (temp->next != NULL) {
            temp->next->prev = temp->prev;
        }
        delete temp;
    }
}

```

```

void search(int key) {
    Node* temp = head;
    int position = 1;
    bool found = false;

```

```
while (temp != NULL) {  
    if (temp->data == key) {  
        std::cout << "Element found at position " << position << std::endl;  
        found = true;  
        break;  
    }  
    temp = temp->next;  
    position++;  
}  
if (!found) {  
    std::cout << "Element not found in the list" << std::endl;  
}  
}
```

```
void display() {  
    if (head == NULL) {  
        std::cout << "List is empty" << std::endl;  
        return;  
    }  
    Node* temp = head;  
    while (temp != NULL) {  
        std::cout << temp->data << " ";  
        temp = temp->next;  
    }  
    std::cout << std::endl;  
}
```

```
};
```

```
int main() {  
    DoublyLinkedList l1;  
    l1.insertAtBeginning(4);  
    l1.insertAtBeginning(6);  
    l1.insertAtBeginning(9);  
    l1.display();  
    l1.deleteAtBeginning();  
    l1.display();  
  
    return 0;  
}
```

CIRCULAR LINK LIST

```
#include <iostream>
```

```
class Node {  
public:  
    int data;  
    Node* next;  
  
    Node(int data) {  
        this->data = data;  
        next = nullptr;  
    }  
}
```

```
};
```

```
class CircularLinkedList {
```

```
private:
```

```
    Node* head;
```

```
public:
```

```
    CircularLinkedList() {
```

```
        head = nullptr;
```

```
    }
```

```
void insertAtLast(int data) {
```

```
    Node* newNode = new Node(data);
```

```
    if (head == nullptr) {
```

```
        head = newNode;
```

```
        head->next = head;
```

```
    } else {
```

```
        Node* temp = head;
```

```
        while (temp->next != head) {
```

```
            temp = temp->next;
```

```
        }
```

```
        temp->next = newNode;
```

```
        newNode->next = head;
```

```
    }
```

```
}
```

```
void insertAtFirst(int data) {
```

```
    Node* newNode = new Node(data);
```

```
    if (head == nullptr) {
```

```
        head = newNode;
```

```
        head->next = head;
```

```
    } else {
```

```
        Node* temp = head;
```

```
        while (temp->next != head) {
```

```
            temp = temp->next;
```

```
        }
```

```
        temp->next = newNode;
```

```
        newNode->next = head;
```

```
        head = newNode;
```

```
    }
```

```
}
```

```
void insertAtIndex(int data, int index) {
```

```
    Node* newNode = new Node(data);
```

```
    if (index <= 0) {
```

```
        std::cout << "Invalid index" << std::endl;
```

```
        return;
```

```
    } else if (index == 1) {
```

```
        insertAtFirst(data);
```

```
    } else {
```

```
        Node* temp = head;
```

```
        for (int i = 1; i < index - 1; i++) {
```



```

        temp = temp->next;
        if (temp == head) {
            std::cout << "Index out of range" << std::endl;
            return;
        }
    }
    newNode->next = temp->next;
    temp->next = newNode;
}
}

```

```

void deleteAtLast() {
    if (head == nullptr) {
        std::cout << "List is empty, nothing to delete" << std::endl;
        return;
    }
    Node* temp = head;
    Node* prev = nullptr;
    while (temp->next != head) {
        prev = temp;
        temp = temp->next;
    }
    if (prev != nullptr) {
        prev->next = head;
    } else {
        head = nullptr;
    }
}

```

```
    }  
    delete temp;  
}
```

```
void deleteAtFirst() {  
    if (head == nullptr) {  
        std::cout << "List is empty, nothing to delete" << std::endl;  
        return;  
    }  
    Node* temp = head;  
    while (temp->next != head) {  
        temp = temp->next;  
    }  
    if (temp == head) {  
        head = nullptr;  
    } else {  
        temp->next = head->next;  
        delete head;  
        head = temp->next;  
    }  
}
```

```
void deleteAtIndex(int index) {  
    if (head == nullptr) {  
        std::cout << "List is empty, nothing to delete" << std::endl;  
        return;  
    }  
}
```

```

    }

    if (index <= 0) {
        std::cout << "Invalid index" << std::endl;
        return;
    }

    if (index == 1) {
        deleteAtFirst();
    } else {
        Node* temp = head;
        Node* prev = nullptr;
        for (int i = 1; i < index; i++) {
            prev = temp;
            temp = temp->next;
            if (temp == head) {
                std::cout << "Index out of range" << std::endl;
                return;
            }
        }
        prev->next = temp->next;
        delete temp;
    }
}

```

```

void search(int key) {
    if (head == nullptr) {
        std::cout << "List is empty" << std::endl;
    }
}

```

```

        return;
    }
    Node* temp = head;
    int position = 1;
    bool found = false;
    do {
        if (temp->data == key) {
            std::cout << "Element found at position " << position << std::endl;
            found = true;
            break;
        }
        temp = temp->next;
        position++;
    } while (temp != head);
    if (!found) {
        std::cout << "Element not found in the list" << std::endl;
    }
}

```

```

void display() {
    if (head == nullptr) {
        std::cout << "List is empty" << std::endl;
        return;
    }
    Node* temp = head;
    do {

```

```
        std::cout << temp->data << " ";  
        temp = temp->next;  
    } while (temp != head);  
    std::cout << std::endl;  
}  
};
```

```
int main() {  
    CircularLinkedList cll;  
    int choice, element, index, searchKey;  
  
    do {  
        std::cout << "\nCircular Linked List Operations:" << std::endl;  
        std::cout << "1. Insert at last" << std::endl;  
        std::cout << "2. Insert at first" << std::endl;  
        std::cout << "3. Insert at specific index" << std::endl;  
        std::cout << "4. Delete at last" << std::endl;  
        std::cout << "5. Delete at first" << std::endl;  
        std::cout << "6. Delete at specific index" << std::endl;  
        std::cout << "7. Search for element" << std::endl;  
        std::cout << "8. Display list" << std::endl;  
        std::cout << "9. Exit" << std::endl;  
        std::cout << "Enter your choice: ";  
        std::cin >> choice;  
  
        switch (choice) {
```

case 1:

```
std::cout << "Enter element to insert at last: ";  
std::cin >> element;  
cll.insertAtLast(element);  
break;
```

case 2:

```
std::cout << "Enter element to insert at first: ";  
std::cin >> element;  
cll.insertAtFirst(element);  
break;
```

case 3:

```
std::cout << "Enter element to insert: ";  
std::cin >> element;  
std::cout << "Enter index to insert at: ";  
std::cin >> index;  
cll.insertAtIndex(element, index);  
break;
```

case 4:

```
ccl.deleteAtLast();  
break;
```

case 5:

```
ccl.deleteAtFirst();  
break;
```

case 6:

```
std::cout << "Enter index to delete: ";  
std::cin >> index;
```

```
        cll.deleteAtIndex(index);

        break;

    case 7:

        std::cout << "Enter element to search: ";

        std::cin >> searchKey;

        cll.search(searchKey);

        break;

    case 8:

        std::cout << "Circular Linked List: ";

        cll.display();

        break;

    case 9:

        std::cout << "Exiting program...";

        break;

    default:

        std::cout << "Invalid choice. Please try again." << std::endl;

    }

} while (choice != 9);

return 0;

}
```